

PHP Form Validation

José Manuel Sánchez Álvarez

Contents

1	Form Validation	1
1.0.1	Basic Steps for PHP Form Validation:	1
1.0.2	Example: Simple PHP Form Validation	2
1.0.3	Conclusion	3

1 Form Validation

Form validation is a crucial aspect of web development to ensure that the data collected from users is correct and useful. In PHP, form validation involves checking the data submitted by the user against certain criteria before processing it. Here's an overview of how to perform basic form validation in PHP:

1.0.1 Basic Steps for PHP Form Validation:

1. **Check if the Form is Submitted:** First, check whether the form has been submitted. This can typically be done by checking if the form's submit button has been clicked.

```
<?
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Form was submitted
}
```

2. **Sanitize Input Data:** Before validating, it's important to sanitize the input to prevent security issues like SQL injections and cross-site scripting (XSS). PHP provides several functions for this, such as `trim()`, `stripslashes()`, and `htmlspecialchars()`.

```
<?
$name = htmlspecialchars(stripslashes(trim($_POST["name"])));
```

3. **Validate Each Field:** Check each input against specific criteria. For example, if a field must be filled out, check if it's empty; if an email is expected, check if it's a valid email format.

```
<?
if (empty($_POST["email"])) {
    $emailErr = "Email is required";
} else {
    $email = test_input($_POST["email"]);
```

```

        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            $emailErr = "Invalid email format";
        }
    }
}

```

4. **Return Error Messages:** If validation fails, return a meaningful error message to the user. This helps them understand what corrections are needed.
5. **Process the Data:** Once all fields are validated, you can proceed with processing the data (e.g., storing it in a database, sending an email, etc.).

1.0.2 Example: Simple PHP Form Validation

Here's a simple example of a PHP form with validation for a name and an email field:

```

<?php
$nameErr = $emailErr = "";
$name = $email = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    if (empty($_POST["name"])) {
        $nameErr = "Name is required";
    } else {
        $name = test_input($_POST["name"]);
        // check if name only contains letters and whitespace
        if (!preg_match("/^[a-zA-Z-' ]*$/", $name)) {
            $nameErr = "Only letters and white space allowed";
        }
    }
}

if (empty($_POST["email"])) {
    $emailErr = "Email is required";
} else {
    $email = test_input($_POST["email"]);
    // check if e-mail address is well-formed
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $emailErr = "Invalid email format";
    }
}
}

function test_input($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
}

```

```
?>

<form method="post"
  action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
  Name: <input type="text" name="name" value="<?php echo $name;?>">
  <span class="error">* <?php echo $nameErr;?></span>
  <br><br>
  E-mail: <input type="text" name="email"
           value="<?php echo $email;?>">
  <span class="error">* <?php echo $emailErr;?></span>
  <br><br>
  <input type="submit" name="submit" value="Submit">
</form>
```

In this example, the `test_input` function sanitizes the data. The name and email fields are validated for being required, and additional checks are performed for proper formatting.

1.0.3 Conclusion

Form validation in PHP is essential for ensuring data integrity and security. While the above example covers basic validation, real-world applications may require more complex checks and security measures, such as validating against a list of allowed values, checking for uniqueness in a database, or using CAPTCHA for spam prevention.